Can LLMs Generate Valid LLVM IR Test Cases for Differential Compiler Testing?

Assignment 18 | Compiler Testing & Fuzzing

June 2026

Meghana J L - 1RV23IS070
Mithra N Gowda - 1RV23IS071

Abstract

This report investigates the capability of large language models (LLMs) to generate and mutate structurally valid LLVM Intermediate Representation (IR) test cases for differential compiler testing. Using a custom-built web-based prototype — LLVM DiffTester — we conducted 38 controlled experiments across two distinct LLM backends: Gemini 2.5 Flash and ChatGPT (GPT-4o). The system compiled mutated IR under two LLVM optimisation levels (opt -O0 and opt -O3) and compared their outputs to identify miscompilation. Results indicate a strong model-dependency in IR validity, with GPT-4o achieving a 100% validity rate (18/18) against Gemini 2.5 Flash achieving 55% (11/20). One potential compiler miscompilation was detected. The findings suggest LLMs can serve as a viable — but model-sensitive — complement to traditional grammar-based fuzzing tools such as Csmith and YARPGen.

1. Introduction

Compiler correctness is a foundational concern in software engineering. A miscompilation — where a semantically correct source program is transformed into an executable with different observable behaviour — can silently corrupt computationally critical systems, from operating system kernels to safety-critical embedded firmware. The LLVM compiler infrastructure is widely deployed in industry and research, making the quality of its testing directly relevant to software reliability at scale.

LLVM's existing testing ecosystem is mature. Tools such as Csmith and YARPGen generate syntactically and semantically valid C programs for differential testing between compiler versions or optimisation levels. Coverage-guided fuzzers such as libFuzzer operate at the binary level, maximising code coverage through feedback-directed mutation. Despite these advances, generating high-quality LLVM IR directly — rather than through a C front-end — remains a difficult problem. LLVM IR is governed by strict structural rules: Static Single Assignment (SSA) form, dominance constraints, typed operands, correct PHI node placement, and precise semantics for constructs such as poison and undef. Naive mutations of IR text almost universally yield invalid IR that is rejected before reaching any optimiser.

This report evaluates whether large language models, prompted with explicit mutation tasks and structural constraints, can produce valid and semantically interesting LLVM IR test cases. The evaluation is grounded in empirical data collected through 38 test runs on a purpose-built differential testing platform.

2. System Design and Methodology

To conduct a reproducible empirical evaluation, a web-based differential testing platform (LLVM DiffTester) was developed using Flask (Python), SQLite, and vanilla JavaScript. The system implements a five-stage pipeline:

- **Stage 1 — Seed IR Generation:** A user uploads a .c source file. The system invokes clang -S -emit-llvm -O0 to produce the corresponding LLVM IR, which serves as the mutation seed.
- **Stage 2 — Prompt-Guided LLM Mutation:** The seed IR is embedded into one of six structured prompt templates corresponding to different mutation strategies: add arithmetic instruction, change integer types, insert conditional branch, insert PHI node, insert dead code, and swap/replace operands. The complete prompt is presented to the user for submission to an external LLM.
- **Stage 3 — Validation Filtering:** The user pastes the LLM-returned IR back into the system. The IR is submitted to llvm-as and opt --verify. Failures are classified into one of six error categories (SSA_VIOLATION, TYPE_MISMATCH, INVALID_PHI, DOMINANCE_ERROR, MISSING_TERMINATOR, UNKNOWN_ERROR) and logged to a persistent SQLite database.
- **Stage 4 — Differential Testing:** Valid IR is compiled under two optimisation pipelines — opt -O0 and opt -O3 — and executed via lli. Standard output and exit codes are compared. Any divergence is flagged as a potential miscompilation.
- **Stage 5 — Analysis and Reporting:** All results are aggregated in real time on an analysis dashboard displaying validity rates, error distributions, per-mutation-type effectiveness, and a timeline of test activity.

2.1 Mutation Types Evaluated

Six mutation strategies were tested across both LLM backends. Each strategy targets a distinct structural dimension of LLVM IR, thereby exercising different families of LLVM optimisation passes: instruction combining (add arithmetic), type legalisation (change types), control-flow graph restructuring (insert branch), loop analysis (insert PHI), dead code elimination (dead code), and instruction canonicalisation (swap operands).

3. Experimental Results

3.1 Overall Validity by LLM Model

A total of 38 test runs were conducted: 20 using Gemini 2.5 Flash and 18 using ChatGPT (GPT-4o). The aggregate validity rate across all runs was 76.3% (29 of 38 cases). The results varied substantially by model, as detailed in Table 1.

LLM Model	Test Cases	Valid (Passed)	Invalid (Failed)	Validity Rate
Gemini 2.5 Flash	20	11	9	55.0%
ChatGPT (GPT-4o)	18	18	0	**100.0%**
Combined	**38**	**29**	**9**	**76.3%**

ChatGPT (GPT-4o) **achieved a 100% validity rate** across all 18 test cases, producing structurally correct IR for every mutation type attempted. **Gemini 2.5 Flash**, by contrast, produced valid IR in only 11 of 20 cases (55.0%), with 9 cases rejected by the LLVM verifier. This differential — 45 percentage points in validity rate between the two models — is the single most significant finding of this study and demonstrates that LLM suitability for IR mutation is strongly model-dependent.

3.2 Error Type Distribution

Among the 9 invalid cases produced by Gemini 2.5 Flash, the dominant failure category was UNKNOWN_ERROR (7 cases), typically involving malformed metadata blocks, conflicting function attributes, or IR constructs that llvm-as rejected without a clear pattern match to known error categories. SSA violations and type mismatches each accounted for one case. Table 2 presents the complete error breakdown.

Error Category	Gemini 2.5 Flash	ChatGPT (GPT-4o)	Root Cause
UNKNOWN_ERROR	7	0	Metadata / attribute conflicts
SSA_VIOLATION	1	0	Undefined SSA name referenced
TYPE_MISMATCH	1	0	Mismatched operand types
Total Invalid	**9**	**0**	

The prevalence of UNKNOWN_ERROR suggests that Gemini 2.5 Flash, while capable of understanding high-level mutation intent, produced IR containing subtle structural inconsistencies — particularly in metadata and attribute sections — that the LLVM verifier rejected. These are precisely the failure modes that are difficult to address through simple rule-based post-processing filters, and they highlight the importance of pairing LLM output with a verifier-based gate.

3.3 Differential Testing Outcome

Of the 29 valid test cases that passed verification, **28 produced identical output under both opt -O0 and opt -O3**, confirming semantic equivalence across optimisation levels. **One case produced a divergent output** (O0/O3 mismatch), representing a potential miscompilation. This case was flagged as an interesting case in the system database and preserved for further analysis. The overall mismatch rate among valid cases was 3.4% (1 of 29).

It should be noted that not all output mismatches necessarily represent compiler bugs. Cases where the mutated IR contains undefined behaviour — for instance, reliance on the value of undef operands — permit the optimiser to produce any result. Confirming a genuine miscompilation requires manual review and, where applicable, reduction using tools such as llvm-reduce or creduce.

3.4 Validity by Mutation Type

Across both models, add arithmetic instruction and swap/replace operands consistently yielded the highest validity rates (approaching 100% for GPT-4o), as these mutations require only localised SSA-preserving transformations. Insert PHI node and insert dead code showed the lowest rates under Gemini 2.5 Flash (60% and 33% respectively), consistent with the expectation that control-flow and loop-level mutations impose the greatest structural complexity for an LLM to manage correctly.

4. Comparison with Traditional Fuzzing Approaches

Table 3 situates LLM-based IR mutation within the broader landscape of compiler testing tools. The comparison highlights both the relative strengths and the current limitations of LLM-guided approaches.

Criterion	LLM-based Mutation	Csmith / YARPGen	Coverage-Guided Fuzzing
Output Validity Rate	55–100% (model-dependent)	Near 100% (grammar enforced)	~90–99% (verifier gated)
Semantic Diversity	High — novel patterns possible	Medium — grammar-constrained	Medium — mutation-bounded
Structural Correctness	Variable — SSA errors common	High — by construction	High — verifier-filtered
Human Prompt Effort	Low (prompt template reuse)	High (grammar authoring)	Medium (seed corpus needed)
Speed / Throughput	Low (API latency)	Very high (native binary)	High (coverage loop)
Bug Discovery (observed)	1 mismatch in 38 runs (2.6%)	Historically effective	Historically effective

Traditional grammar-based generators such as Csmith and YARPGen produce valid programs by construction — they generate source code that conforms to the C language grammar, ensuring near-100% validity prior to compilation. Coverage-guided fuzzers such as AFL++ or libFuzzer operating on LLVM IR bitcode achieve similar reliability through tight feedback loops with the verifier. LLM-based mutation, by contrast, treats validity as an emergent property of the model's

learned representation of IR syntax and semantics — a property that is model-dependent and inconsistent without post-generation verification.

However, LLM-based mutation offers a qualitatively different form of diversity. Grammar-based tools are bounded by the expressiveness of their production rules; they cannot, without explicit extension, generate mutation patterns that their authors did not anticipate. LLMs, drawing on broad training exposure to compiler literature, IR specifications, and code repositories, are capable of producing novel structural combinations — such as non-trivial PHI node configurations or mixed-type arithmetic chains — that may exercise optimiser paths that grammar-bounded generators systematically miss.

The principal practical limitation of LLM-based mutation is throughput. API latency for a single mutation request ranges from one to several seconds, compared to the microsecond-level generation speed of native binary fuzzers. For large-scale fuzzing campaigns, this difference is prohibitive. LLM mutation is therefore best understood not as a replacement for coverage-guided fuzzing, but as a targeted complement for human-directed exploration of specific mutation hypotheses.

5. Discussion

The results of this study permit a nuanced answer to the central research question: LLMs can generate valid and semantically meaningful LLVM IR test cases, but their utility is highly dependent on the capability of the underlying model, and they should not be deployed without a verifier-based filter.

The 100% validity rate achieved by GPT-4o is notable. It suggests that sufficiently capable models, given well-structured prompts that explicitly enumerate the SSA and type constraints the output must satisfy, can reliably produce verifier-correct IR mutations. The prompt templates used in this study — which specified mutation goals, SSA naming requirements, type consistency rules, and dominance constraints in natural language — appear to have been sufficient for GPT-4o but insufficient for Gemini 2.5 Flash.

This observation has a practical implication: the quality of LLM-based IR mutation is not solely a function of the prompt design; it is bounded by the model's internal representation of LLVM IR semantics. More capable models may internalise these constraints more reliably. As frontier LLM capabilities continue to advance, the validity gap between models such as Gemini 2.5 Flash and GPT-4o may narrow, making LLM-based IR mutation increasingly practical across a wider range of model backends.

The detection of one O0/O3 output mismatch in 38 runs (2.6% of total runs, 3.4% of valid runs) is preliminary but encouraging. A sustained campaign of several hundred runs with multiple mutation types and seed programs would be necessary to draw statistically robust conclusions about the

bug-finding rate relative to traditional fuzzers. Nevertheless, the fact that a mismatch was detected at all — from a small, manually operated experiment — supports the hypothesis that LLM-generated mutations can exercise real optimiser paths.

The dominance of UNKNOWN_ERROR in Gemini 2.5 Flash's failure cases also points to a potential area for improvement. A post-processing step that strips or normalises IR metadata before submission to the verifier could eliminate a significant proportion of false negatives — cases where the mutation logic itself is correct but the surrounding metadata is malformed. Such a normalisation pass would be straightforward to implement and could substantially improve Gemini 2.5 Flash's effective validity rate.

6. Conclusion

This study demonstrates that LLM-based LLVM IR mutation is a technically viable approach to compiler differential testing, subject to three conditions: (1) the LLM backend must have sufficient capability to produce verifier-correct IR, (2) prompts must explicitly communicate structural constraints in natural language, and (3) all LLM output must be filtered through a verifier gate before use in differential testing.

Across 38 empirical test runs, GPT-4o achieved a perfect validity rate while Gemini 2.5 Flash achieved 55%, yielding an aggregate validity rate of 76.3%. One potential miscompilation was identified. LLM-based mutation is not a replacement for established tools such as Csmith, YARPGen, or coverage-guided fuzzers, which operate at substantially higher throughput and with near-100% structural validity. It is, however, a promising complement — capable of generating semantically diverse mutations that grammar-bounded generators may systematically miss.

Future work should investigate automated post-processing pipelines to repair common LLM-produced IR errors, evaluate additional LLM models (including fine-tuned code models), and conduct longitudinal fuzzing campaigns with larger seed corpora to establish statistically grounded comparisons with traditional fuzzing bug-discovery rates.